APOGEE: Planar Two-Stage Ascent to Circular LEO (Mathematical Formulation and Numerical Methods; Code-Consistent)

Abstract

This report derives, from first principles, the exact mathematical model and numerical methods implemented in the current APOGEE codebase (`packages/apogee-core/src/apogee_core`). The simulator propagates a planar (2D) Earth-centered ascent with variable mass, non-constant gravity, US Standard Atmosphere 1976 (USSA76) density and speed of sound, and aerodynamic drag. Flight is modeled as a hybrid dynamical system with event-driven phase changes (pitch-over, stage-1 burnout and separation, coast, stage-2 burn termination by time or fuel depletion, and ground impact).

The circular-orbit insertion problem is solved via a shooting method that adjusts a four-dimensional control vector $\boldsymbol{u} = (\theta_0, t_{\text{coast}}, t_{\text{burn2}}, \alpha_2)$ and drives a three-component residual based on terminal osculating orbital elements: $\boldsymbol{F}(\boldsymbol{u}) = [(a - r_{\text{target}})/r_{\text{target}},\ e,\ \gamma]^T$. The nonlinear solve is performed with a Levenberg–Marquardt-type iteration using a finite-difference Jacobian in an unconstrained variable $\boldsymbol{x} \in \mathbb{R}^4$ (logistic re-parameterization to enforce bounds), a Broyden rank-1 update, and a backtracking line search.

Every equation and every numerical method is explicitly mapped to the corresponding implementation function and file, to avoid ambiguity.

1 Problem statement

1.1 Mission inputs

The simulator mission is parameterized by:

- Target circular orbit altitude h_{target} (m).
- Payload mass m_{PL} (kg).

Define

$$r_{\text{target}} = R_E + h_{\text{target}}, \tag{1}$$

where R_E is the Earth reference radius.

1.2 Assumptions

- Planar (2D) inertial motion; Earth rotation and out-of-plane dynamics are neglected.
- Central gravity with parameter μ.
- Two powered stages with constant thrust T and constant specific impulse I_{sp} per stage.
- Aerodynamic drag with $\rho(h)$ and $a_s(h)$ from USSA76.
- Hybrid events define phase transitions.

2 State, geometry, and kinematics

2.1 State variables

We define (planar polar coordinates in an inertial plane):

- $r(t)$: geocentric radius.
- $\lambda(t)$: downrange central angle (rad).
- $v(t)$: speed magnitude.
- $\gamma(t)$: flight-path angle (FPA), measured from local horizontal.
- $m(t)$: vehicle mass.

Altitude is

$$h(t) = r(t) - R_E. \tag{2}$$

The state vector is

$$\boxed{\boldsymbol{y}(t) = \begin{bmatrix} r(t) \\ \lambda(t) \\ v(t) \\ \gamma(t) \\ m(t) \end{bmatrix}}. \tag{3}$$

Code: `apogee_core/dynamics.py` (indexing constants `_R, _LAM, _V, _GAMMA, _M`).

2.2 Velocity decomposition

In planar polar coordinates,

$$\boldsymbol{v} = \dot{r}\,\hat{\boldsymbol{e}}_r + r\dot{\lambda}\,\hat{\boldsymbol{e}}_\lambda. \tag{4}$$

Define γ such that

$$\dot{r} = v\sin\gamma, \qquad r\dot{\lambda} = v\cos\gamma. \tag{5}$$

Therefore the kinematic ODEs are

$$\boxed{\dot{r} = v\sin\gamma}, \qquad \boxed{\dot{\lambda} = \frac{v\cos\gamma}{r}}. \tag{6}$$

Code: `apogee_core/dynamics.py::rhs_general`.

3 Environment model (USSA76)

3.1 Tabulation

USSA76 provides density ρ and speed of sound a_s as functions of geometric altitude h. The code builds a discrete table

$$z_i = i\,\Delta z, \qquad i = 0, \ldots, N, \qquad z_N \approx z_{\max}, \tag{7}$$

and evaluates

$$\rho_i = \rho(z_i), \qquad a_{s,i} = a_s(z_i). \tag{8}$$

Code: `apogee_core/atmosphere.py::build_atmosphere_table` (cached with LRU).

3.2 Interpolation and physically safe extrapolation

During integration, the model uses linear interpolation

$$\rho(h) = \text{interp}(h; \{(z_i, \rho_i)\}), \qquad a_s(h) = \text{interp}(h; \{(z_i, a_{s,i})\}). \tag{9}$$

To avoid non-physical extrapolation above $z_{\max}$, the table is extended with one additional point

$$z_{N+1} = z_N + \Delta z, \qquad \rho_{N+1} = 0, \qquad a_{s,N+1} = a_{s,N}, \tag{10}$$

so that the interpolator has a C^0-continuous approach to vacuum. **Code:** `apogee_core/atmosphere.py::_build_a`

4 Aerodynamic model

Define Mach number

$$M = \frac{v}{a_s(h)}. \tag{11}$$

Given a drag coefficient model $C_D(M)$ and reference area A_{ref}, drag magnitude is

$$\boxed{D = \tfrac{1}{2}\,\rho(h)\,C_D(M)\,A_{\text{ref}}\,v^2.} \tag{12}$$

Dynamic pressure is

$$\boxed{q = \tfrac{1}{2}\,\rho(h)\,v^2.} \tag{13}$$

Code: `apogee_core/dynamics.py::compute_derived`.

5 Propulsion and variable mass

Specific impulse is defined by

$$I_{sp} = \frac{T}{\dot{m}_{\text{prop}} g_0}. \tag{14}$$

With $\dot{m} = -\dot{m}_{\text{prop}}$, mass flow is

$$\boxed{\dot{m} = -\frac{T}{I_{sp} g_0}.} \tag{15}$$

Coast is implemented by setting $T = 0$ and forcing $\dot{m} = 0$. **Code:** `apogee_core/dynamics.py::rhs_general`, `rhs_coast`.

6 Dynamics: derivation of the implemented ODE

6.1 Force model and thrust direction

Gravity acceleration is central:

$$\boldsymbol{a}_g = -\frac{\mu}{r^2}\hat{\boldsymbol{e}}_r. \tag{16}$$

Let $\hat{\boldsymbol{t}}$ be the unit tangent along velocity and $\hat{\boldsymbol{n}}$ the in-plane normal:

$$\hat{\boldsymbol{t}} = \sin\gamma\,\hat{\boldsymbol{e}}_r + \cos\gamma\,\hat{\boldsymbol{e}}_\lambda, \qquad \hat{\boldsymbol{n}} = \cos\gamma\,\hat{\boldsymbol{e}}_r - \sin\gamma\,\hat{\boldsymbol{e}}_\lambda. \tag{17}$$

Let α be the steering angle between thrust and velocity (in-plane). Then

$$\hat{\boldsymbol{u}}_T = \cos\alpha\,\hat{\boldsymbol{t}} + \sin\alpha\,\hat{\boldsymbol{n}}, \qquad \boldsymbol{a}_T = \frac{T}{m}\hat{\boldsymbol{u}}_T. \tag{18}$$

Drag acceleration is opposite velocity:

$$\boldsymbol{a}_D = -\frac{D}{m}\hat{\boldsymbol{t}}. \tag{19}$$

6.2 Speed equation

By definition $\dot{v} = \boldsymbol{a}\cdot\hat{\boldsymbol{t}}$, where $\boldsymbol{a} = \boldsymbol{a}_T + \boldsymbol{a}_D + \boldsymbol{a}_g$. Using

$$\boldsymbol{a}_T\cdot\hat{\boldsymbol{t}} = \frac{T}{m}\cos\alpha, \qquad \boldsymbol{a}_D\cdot\hat{\boldsymbol{t}} = -\frac{D}{m}, \qquad \boldsymbol{a}_g\cdot\hat{\boldsymbol{t}} = -\frac{\mu}{r^2}\sin\gamma, \tag{20}$$

we obtain

$$\boxed{\dot{v} = \frac{T}{m}\cos\alpha - \frac{D}{m} - \frac{\mu}{r^2}\sin\gamma.} \tag{21}$$

6.3 Flight-path angle equation

A standard planar identity gives

$$\boldsymbol{a}\cdot\hat{\boldsymbol{n}} = v\dot{\gamma} - \frac{v^2}{r}\cos\gamma. \tag{22}$$

Compute

$$\boldsymbol{a}_T\cdot\hat{\boldsymbol{n}} = \frac{T}{m}\sin\alpha, \qquad \boldsymbol{a}_D\cdot\hat{\boldsymbol{n}} = 0, \qquad \boldsymbol{a}_g\cdot\hat{\boldsymbol{n}} = -\frac{\mu}{r^2}\cos\gamma. \tag{23}$$

Thus

$$\frac{T}{m}\sin\alpha - \frac{\mu}{r^2}\cos\gamma = v\dot{\gamma} - \frac{v^2}{r}\cos\gamma, \tag{24}$$

and

$$\boxed{\dot{\gamma} = \frac{T}{mv}\sin\alpha + \left(\frac{v}{r} - \frac{\mu}{r^2 v}\right)\cos\gamma.} \tag{25}$$

6.4 Complete ODE system (code-consistent)

Combining kinematics, dynamics, and mass flow:

$$\dot{r} = v\sin\gamma, \tag{26}$$

$$\dot{\lambda} = \frac{v\cos\gamma}{r}, \tag{27}$$

$$\dot{v} = \frac{T}{m}\cos\alpha - \frac{D}{m} - \frac{\mu}{r^2}\sin\gamma, \tag{28}$$

$$\dot{\gamma} = \frac{T}{mv}\sin\alpha + \left(\frac{v}{r} - \frac{\mu}{r^2 v}\right)\cos\gamma, \tag{29}$$

$$\dot{m} = -\frac{T}{I_{sp}g_0}. \tag{30}$$

Code: `apogee_core/dynamics.py::rhs_general`.

7 Numerical regularizations implemented in the RHS

Two numerical guard terms are implemented to avoid singularities and unstable divisions.

7.1 Guard for $1/v$

The $\dot{\gamma}$ equation contains $1/v$. Near lift-off $v \to 0$, this is numerically singular. The implementation replaces

$$\frac{1}{v} \;\rightsquigarrow\; \frac{v}{v^2 + v_\varepsilon^2}, \tag{31}$$

which matches $1/v$ for $v \gg v_\varepsilon$ and remains bounded at $v = 0$. **Code:** `apogee_core/dynamics.py::rhs_general` (variable `inv_v`).

7.2 Guard for $1/r$

The kinematic and dynamic terms include $1/r$ and $1/r^2$. To avoid numerical collapse if r becomes non-physical, the code uses

$$r_{\text{safe}} = \max(r, 0.99 R_E) \tag{32}$$

in denominator terms. **Code:** `apogee_core/dynamics.py::rhs_general` (variable `r_safe`).

8 Hybrid flight phases and event conditions

The full ascent is a hybrid system assembled from multiple ODE segments.

8.1 Phase A: vertical ascent

To avoid the $1/v$ issue at ignition, the vertical phase enforces

$$\gamma = \frac{\pi}{2}, \qquad \dot{\gamma} = 0. \tag{33}$$

The pitch-over event triggers when altitude reaches h_{po}:

$$g_{\text{po}}(\boldsymbol{y}) = (r - R_E) - h_{\text{po}} = 0. \tag{34}$$

After the event, an instantaneous reset sets

$$\gamma^+ = \frac{\pi}{2} - \theta_0. \tag{35}$$

Code: `apogee_core/simulate.py::_cond_pitch_over` and `simulate_ascent`.

8.2 Phase B: stage-1 powered gravity turn

During stage 1, thrust is aligned with velocity:

$$\alpha = 0. \tag{36}$$

Stage-1 burnout is modeled by a mass event. First compute propellant consumed during stage 1

$$m_{1,\text{prop}} = \frac{T_1\, t_{1,\text{burn}}}{I_{sp,1} g_0}, \tag{37}$$

and define end-of-burn mass

$$m_{1,\text{end}} = m_0 - m_{1,\text{prop}}. \tag{38}$$

The burnout event is

$$g_{\text{s1}}(\boldsymbol{y}) = m - m_{1,\text{end}} = 0. \tag{39}$$

Upon separation, a dry-mass drop is applied:

$$m^+ = m^- - m_{1,\text{dry}}. \tag{40}$$

Code: `apogee_core/simulate.py::_cond_stage1_burnout` and separation logic in `simulate_ascent`.

8.3 Phase C: coast

Coast integrates the same ODE with $T = 0$ and enforces $\dot{m} = 0$. **Code:** `apogee_core/dynamics.py::rhs_coast` and `simulate_ascent`.

8.4 Phase D: stage-2 burn with constant steering

Stage 2 uses a constant steering angle

$$\alpha = \alpha_2. \tag{41}$$

The integration stops at either

- fixed burn duration t_{burn2}, or
- fuel depletion event at $m = m_{\text{min}}$.

The physical minimum mass is

$$m_{\text{min}} = m_{2,\text{dry}} + m_{\text{PL}}, \tag{42}$$

and the event is

$$g_{\text{fuel}}(\boldsymbol{y}) = m - m_{\text{min}} = 0. \tag{43}$$

Code: `apogee_core/simulate.py::_cond_fuel_depletion` and stage-2 RHS term `_term_rhs_stage2_steer`.

8.5 Ground impact

Ground impact is detected by

$$g_{\text{ground}}(\boldsymbol{y}) = r - R_E = 0. \tag{44}$$

Code: `apogee_core/simulate.py::_cond_ground`.

9 ODE solver and event root-finding (numerical methods)

Each segment is integrated with:

- **ODE solver:** Tsitouras 5(4) (`diffrax.Tsit5`).
- **Adaptive step-size controller:** PID controller (`diffrax.PIDController(rtol,atol)`).
- **Event root-finding:** Newton method (`optimistix.Newton(rtol=root_rtol, atol=root_atol)`).
- **Saving strategy:** save initial time and every accepted step (`diffrax.SaveAt(t0=True, steps=True)`).

Code: `apogee_core/simulate.py::_solve_segment`.

10 Trajectory assembly and time monotonicity

Segments are concatenated as

$$t = [t^{(A)}, t^{(B)}_{2:}, t^{(C)}_{2:}, t^{(D)}_{2:}], \tag{45}$$

where 2 : indicates dropping the first sample of each subsequent segment to avoid duplicating boundary points.

Even with this, adaptive solvers and event boundaries can produce consecutive equal times. For physical consistency and frontend consumption, the implementation enforces strict monotonicity

$$t_{k+1} > t_k \quad \forall k, \tag{46}$$

by dropping non-increasing consecutive samples and applying the same mask to all derived series.
Code: `apogee_core/simulate.py::_strictly_increasing_mask` and its application to `ts,ys,mach,drag,q`.

11 Two-body orbital diagnostics at cutoff

Given a terminal state (r, v, γ) and Earth parameter μ, define specific orbital energy

$$\varepsilon = \frac{v^2}{2} - \frac{\mu}{r}, \tag{47}$$

and specific angular momentum magnitude (planar)

$$h = rv\cos\gamma. \tag{48}$$

Eccentricity is computed as

$$\boxed{e = \sqrt{\max\left\{0,\ 1 + \frac{2\varepsilon h^2}{\mu^2}\right\}}.} \tag{49}$$

For bound orbits ($\varepsilon < 0$), the semi-major axis is

$$\boxed{a = -\frac{\mu}{2\varepsilon}.} \tag{50}$$

Apoapsis and periapsis radii are

$$r_a = a(1+e), \qquad r_p = a(1-e). \tag{51}$$

Code: `apogee_core/orbit.py::orbit_diagnostics`.

12 Shooting formulation (current implementation)

12.1 Decision variables

The shooting solver adjusts the control vector

$$\boxed{\boldsymbol{u} = \begin{bmatrix} \theta_0 \\ t_{\text{coast}} \\ t_{\text{burn2}} \\ \alpha_2 \end{bmatrix}.} \tag{52}$$

Code: `apogee_core/shooting.py::solve_circular_orbit` and `compute_residuals`.

12.2 Map from controls to terminal orbit

Given $\boldsymbol{u}$, the code runs a fast terminal-only simulation (same hybrid model) to obtain a terminal state $\boldsymbol{y}_f(\boldsymbol{u})$. **Code:** `apogee_core/simulate.py::simulate_ascent_final`.

From $\boldsymbol{y}_f$ it computes terminal $(a(\boldsymbol{u}), e(\boldsymbol{u}))$ using the two-body formulas.

12.3 Residual definition

The residual implemented is

$$\boxed{\boldsymbol{F}(\boldsymbol{u}) = \begin{bmatrix} \dfrac{a(\boldsymbol{u}) - r_{\text{target}}}{r_{\text{target}}} \\ e(\boldsymbol{u}) \\ \gamma(\boldsymbol{u}) \end{bmatrix}.} \tag{53}$$

Rationale:

- A circular orbit at radius r_{target} has $a = r_{\text{target}}$.
- Circularity requires $e = 0$.
- Tangency at insertion requires $\gamma = 0$.

Code: `apogee_core/shooting.py::compute_residuals`.

13 Numerical method for shooting

13.1 Bound constraints and logistic re-parameterization

Physical bounds are enforced:

$$\theta_0 \in [\theta_{\min}, \theta_{\max}], \quad t_{\text{coast}} \in [0, 200], \quad t_{\text{burn2}} \in [50, 450], \quad \alpha_2 \in [\alpha_{\min}, \alpha_{\max}]. \tag{54}$$

To solve in $\mathbb{R}^4$, introduce $\boldsymbol{x} \in \mathbb{R}^4$ and

$$u_i(x_i) = u_{i,\min} + (u_{i,\max} - u_{i,\min})\, \sigma(x_i), \qquad \sigma(x) = \frac{1}{1 + e^{-x}}. \tag{55}$$

Code: `apogee_core/shooting.py::_u_from_x`, `_x_from_u`.

13.2 Finite-difference Jacobian in $\boldsymbol{x}$

Let $\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{F}(\boldsymbol{u}(\boldsymbol{x}))$. A forward-difference Jacobian is computed as

$$J_{:,i} \approx \frac{\boldsymbol{f}(\boldsymbol{x} + \Delta x_i\, \boldsymbol{e}_i) - \boldsymbol{f}(\boldsymbol{x})}{\Delta x_i}. \tag{56}$$

Code: `apogee_core/shooting.py::_fd_jacobian_x`.

13.3 Levenberg–Marquardt step with backtracking

At iteration k, the code solves the damped normal equations

$$(J_k^T J_k + \lambda_k I)\, \Delta \boldsymbol{x}_k = -J_k^T \boldsymbol{f}_k, \tag{57}$$

tries a finite set of step lengths $\alpha \in \{1, \frac{1}{2}, \frac{1}{4}, \dots\}$, and accepts the first α satisfying strict decrease in $\|\boldsymbol{f}\|_2$. **Code:** `apogee_core/shooting.py::_newton`.

13.4 Broyden rank-1 update

After an accepted step $\boldsymbol{s} = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$ and $\boldsymbol{y} = \boldsymbol{f}_{k+1} - \boldsymbol{f}_k$,

$$J_{k+1} = J_k + \frac{(\boldsymbol{y} - J_k \boldsymbol{s})\boldsymbol{s}^T}{\boldsymbol{s}^T \boldsymbol{s}}. \tag{58}$$

This reduces expensive residual evaluations. **Code:** `apogee_core/shooting.py::_broyden_update`.

13.5 Multistart candidate generation and scoring

Candidates are generated from a seed and coarse grids for each control variable, then scored by $\|\boldsymbol{F}(\boldsymbol{u}_0)\|_2$ to select the best initializations. **Code:** candidate generation in `apogee_core/shooting.py` near `theta0_grid, t_coast_grid, t_burn2_grid, alpha2_grid`.

13.6 Convergence criteria

The implementation checks component-wise tolerances and an overall norm tolerance:

$$|F_1| < \varepsilon_1, \quad |F_2| < \varepsilon_2, \quad |F_3| < \varepsilon_3 \quad \text{or} \quad \|\boldsymbol{F}\|_2 < \varepsilon. \tag{59}$$

Code: `apogee_core/shooting.py::_ok` and `_ok_norm`.

14 Serialization: JSON trajectory output

The function `trajectory_to_dict` returns JSON-serializable arrays with explicit unit suffixes:

- `t_s`, `r_m`, `h_m`, `lambda_rad`, `v_mps`, `gamma_rad`, `m_kg`, `q_pa`, `mach`, `drag_n`.
- `pos_m`: planar (x, y, z) with $z = 0$.
- `vel_mps`: planar (v_x, v_y, v_z) with $v_z = 0$.
- `orbit`: scalar two-body diagnostics.

Code: `apogee_core/trajectory.py::trajectory_to_dict`.

15 Calibration module (implemented, but auxiliary)

The repository also contains a calibration routine in `apogee_core/calibration.py`. It defines an outer optimization problem (SciPy L-BFGS-B) over a small parameter vector (e.g., $I_{sp,1}$, $I_{sp,2}$, $t_{1,\text{burn}}$, constant C_D value), while calling the shooting solver as an inner loop.

This module is not required for running the simulator/shooting pipeline, but it is part of the implemented code and therefore documented here.

16 Equation-to-code map

Mathematical object	Implementation
Atmosphere table, interpolation, vacuum extension	`apogee_core/atmosphere.py`
Derived scalars (h, ρ, a_s, M, D, q)	`apogee_core/dynamics.py::compute_derived`
ODE RHS (general)	`apogee_core/dynamics.py::rhs_general`
Vertical / gravity turn / coast	`rhs_vertical`, `rhs_gravity_turn`, `rhs_coast`
Event functions	`apogee_core/simulate.py::_cond_*`
ODE solve per segment	`apogee_core/simulate.py::_solve_segment`
Time monotonicity filter	`apogee_core/simulate.py::_strictly_increasing_mask`
Orbital diagnostics $(\varepsilon, h, a, e, r_a, r_p)$	`apogee_core/orbit.py`
Shooting residual $[(a - r_t)/r_t,\ e,\ \gamma]$	`apogee_core/shooting.py::compute_residuals`
LM step + Broyden + line search	`apogee_core/shooting.py::_newton`
Trajectory JSON	`apogee_core/trajectory.py::trajectory_to_dict`